

Laboratory work 13

Testing React Apps

This chapter explains the basics of testing React apps. It will give us an overview of using Jest, which is a JavaScript testing framework. We will look at how you can create and run new test suites and tests. To test our React Vite project, we will also learn how to use the React Testing Library together with Vitest.

In this chapter, we will cover the following topics:

- Using Jest
- Using the React Testing Library
- Using Vitest
- Firing events in tests
- End-to-end testing

Technical requirements

The Spring Boot application that we created in *Chapter 5, Securing Your Backend*, is required, as is the React app that we used in *Chapter 14, Styling the Frontend with React MUI*.

The code samples available at the following GitHub link will also be required to follow along with this chapter: <https://github.com/PacktPublishing/Full-Stack-Development-with-SpringBoot-3-and-React-Fourth-Edition/tree/main/Chapter15>.

Using Jest

Jest is a testing framework for JavaScript, developed by Meta Inc. (<https://jestjs.io/>). It is widely used with React and provides lots of useful features for testing. For example, you can create a **snapshot** test, whereby you can take snapshots from React trees and investigate how states are changing. Jest has mocking functionalities that you can use to test, for example, your asynchronous REST API calls. It also provides functions that are required for assertions in your test cases.

To demonstrate the syntax, we will see how to create a test case for a basic TypeScript function that performs some simple calculations. The following function takes two numbers as arguments and returns the product of the numbers:

```
// multi.ts export const calcMulti = (x: number, y: number):  
number => { return x * y;  
}
```

The following code snippet shows a Jest test for the preceding function:

```
// multi.test.ts import { calcMulti } from  
'./multi';  
  
test("2 * 3 equals 6", () => {  
  expect(calcMulti(2, 3)).toBe(6);  
});
```

The test case starts with a `test()` method that runs the test case. The `test()` method takes two required arguments: the test name (a descriptive string) and the anonymous function that contains the test code. The `expect()` function is used when you want to test values, and it gives you access to multiple **matchers**. The `toBe()` function is one matcher that checks whether the result from the function equals the value in the matcher.



There are many different matchers available in Jest, and you can find them in the documentation: <https://jestjs.io/docs/using-matchers>.

`describe()` is a function that's used in test suites to group related test cases together. It helps you to organize tests based on their functionality, or in React, based on the component being tested. In the following example, we have a test suite that contains two test cases for the App component:

```
describe("App component", () => { test("App component  
renders", () => {  
  // 1st test case  
})  
  
  test("Header text", () => {  
    // 2nd test case  
  })  
});
```

Using the React Testing Library

The **React Testing Library** (<https://testing-library.com/>) is a set of tools and APIs for testing React components. It can be used for DOM testing and queries. The React Testing Library provides a set of query functions that help you search elements based on their text content, label, and so on. It also provides tools to simulate user actions such as clicking a button and typing into input fields.

Let's go through some important concepts in the React Testing Library. The Testing Library provides a `render()` method that renders a React element into the DOM and makes it available for testing:

```
import { render } from '@testing-library/react'

render(<MyComponent />);
```

Queries can be used to find elements on the page. The `screen` object is a utility for querying the rendered components. It provides a set of query methods that can be used to find elements on the page. There are different types of queries that start with various keywords: `getBy`, `findBy`, or `queryBy`. The `getBy` and `findBy` queries throw an error if no element is found. The `queryBy` queries return null if no element is found.



The right query to use depends on the situation, and you can read more about the differences at [https://testing-library.com/docs/dom-testing-library / cheatsheet/](https://testing-library.com/docs/dom-testing-library/cheatsheet/).

For example, the `getByText()` method queries the document for an element that contains the specified text:

```
import { render, screen } from '@testing-library/react'

render(<MyComponent />);
// Find text Hello World (case-insensitive) screen.getByText(/Hello World/i);
```

The forward slash (/) in `/Hello World/i` is used to define a regular expression pattern, and the `i`-flag at the end stands for case-insensitive. This means it is looking for rendered content that contains the “Hello World” text in a case-insensitive matter. You can also use a full string match that is case-sensitive by passing a string as an argument:

```
screen.getByText("Hello World");
```

Then, we can use `expect` to make an assertion. `jest-dom` is a companion library for the React Testing Library, and it provides custom matchers that are useful when testing React components.

For example, its `toBeInTheDocument()` matcher checks if the element is present in the document. If the following assertion passes, the test case will pass; otherwise, it will fail:

```
render(<MyComponent />);  
expect(screen.getByText(/Hello World/i)).toBeInTheDocument();
```



You can find all the matchers in the `jest-dom` documentation: <https://github.com/testing-library/jest-dom>

```
import { render, screen } from '@testing-library/react' import matchers from  
'@testing-library/jest-dom/matchers';
```

We have now learned the basics of Jest and the React Testing Library. Both libraries are needed to test React applications. Jest is a testing framework that provides a testing environment and assertion library. The React Testing Library is a utility library designed for testing React components. Next, we will learn how to start testing in a Vite project.

Using Vitest

Vitest (<https://vitest.dev/>) is the testing framework for Vite projects. It is also possible to use Jest in Vite projects, and there are libraries that provide Vite integration for Jest (for example, <https://github.com/sodatea/vite-jest>). In this book, we will use Vitest because it is easier to start using it with Vite. Vitest is similar to Jest, and it provides `test`, `describe`, and `expect`, which we learned about in the Jest section.

In this section, we will create tests with Vitest and the React Testing Library for the frontend project that we used in *Chapter 14, Styling the Frontend with MUI*.

Installing and configuring

The first step is installing Vitest and the React Testing Library to our project:

1. Open the project in Visual Studio Code. Move to your project folder in the terminal and execute the following npm command inside your project folder:

```
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

The `-D` flag in the npm command means that a package should be saved as a development dependency in the `devDependencies` section of the `package.json` file. These packages are necessary for development and testing but are not required for the production runtime of the application.

2. Next, we have to configure Vitest by using a Vite configuration file, `vite.config.ts`. Open the file and add a new test property with the following changes:

```
import { defineConfig } from 'vite/config'
import react from '@vitejs/plugin-react'

// https://vitejs.dev/config/ export
default defineConfig({
  plugins: [react()],    test: {
    globals:
    true,
    environment:
    'jsdom',
  },
})
```

By default, Vitest does not provide a global API. The `globals: true` setting allows us to reference APIs globally (`test`, `expect`, and so on), like Jest. The `environment: 'jsdom'` setting defines that we are using the browser environment instead of Node.js.

3. Now, you can see a TypeScript type error in the `test` property because the `test` type doesn't exist in Vite's configuration. You can import extended Vite configuration from Vitest to get rid of the error. Modify the `defineConfig` import as shown in the following code:

```
// Modify defineConfig import
import { defineConfig } from 'vitest/config'
```

4. Next, we will add the `test` script to our `package.json` file:

```
"scripts": {
  "dev": "vite",
  "build": "tsc && vite build",
  "lint": "eslint src --ext ts,tsx --report-unused-disable-
directives --max-warnings 0",    "preview": "vite preview",
  "test": "vitest"
},
```

5. We can now run our tests using the following npm command. In this phase, you will get an error because we don't have any tests yet:

```
npm run test
```



You can also find a Visual Studio Code extension for Vitest if you want to run your tests from the VS Code IDE:

<https://marketplace.visualstudio.com/items?itemName=ZixuanChen.vitest-explorer>.

By default, files to include in the test run are defined using the following glob pattern (<https://vitest.dev/config/#include>):

```
['**/*.test.{ts,tsx}.?(c|m)[jt]s?(x)']
```

We will name our test files using the `component.test.tsx` naming convention.

Running our first test

Now, we will create our first test case to verify that our App component is rendered and that the app header text can be found:

1. Create a new file called `App.test.tsx` in the `src` folder of your React app and create a new test case. We are using Vitest, so we import `describe` and `test` from `vitest`:

```
import { describe, test } from 'vitest';

describe("App tests", () => {
  test("component renders", () => {
    // Test case code
  })
});
```

2. Then, we can use the render method from the React Testing Library to render our App component:

```
import { describe, test } from 'vitest';
import { render } from '@testing-library/react'; import
App from './App';
describe("App tests", () => { test("component
renders", () => {
render(<App />);
})
});
```

3. Next, we use the screen object and its query API to verify that the app header text has been rendered:

```
import { describe, test, } from 'vitest'; import
{ render, } from '@testing-library/react';
import App from './App';

describe("App tests", () => {
test("component renders", () => {
render(<App />);
expect(screen.getByText(/Car Shop/i)).toBeDefined();
})
});
```

4. If you want to use jest-dom library matchers such as `toBeInTheDocument()`, which we used earlier, you should import the `jest-dom/vitest` package, which extends matchers:

```
import { describe, test, expect } from '@testing-library/jest-dom/vitest';
import { render, screen } from '@testing-library/react'; import App from './App';

describe("App tests", () => {
  test("component renders", () => {
    expect(screen.getByText(/Car Shop/i)).toBeInTheDocument();
  });
});
```

5. Finally, we can run our test by typing the following command in the terminal:

```
npm run test
```

We should see that the test passes:

```
✓ src/App.test.tsx (1)

Test Files  1 passed (1)
Tests       1 passed (1)
Start at    11:07:01
Duration    2.98s (transform 174ms, setup 0ms, collect 1
PASS Waiting for file changes...
press h to show help, press q to quit
```

Figure 15.1: Test run

Tests are run in **watch mode**, meaning each time you make changes to your source code, the tests that are related to the code changes are rerun. You can quit watch mode by pressing *q*, as shown in the figure. You can also invoke test reruns manually by pressing *r*.

If you need, you can create a test setup file that can be used to set up the environment and configuration required for running tests. The setup file will be run before each test file.

You have to specify the path to the test setup file in the `vite.config.ts` file, inside the `test` node:

```
// vite.config.ts
test: {
  setupFiles: ['./src/testSetup.ts'],
  globals: true,
  environment:
```




```
'jsdom',
},
```

You can also perform tasks that are required before or after test cases. Vitest provides the `beforeEach` and `afterEach` functions that you can use to invoke code before or after your test cases. For example, you can run the React Testing Library's `cleanup` function after each test case to unmount React components that were mounted. If

you only want to invoke some code once before or after *all* test cases, you can use the `beforeAll` or `afterAll` functions.

Testing our Carlist component

Let's now make a test for our `Carlist` component. We will use our backend REST API, and in this section, you should run the backend that we used in the previous chapter. Using a real API in your tests is closer to a real-world scenario and allows end-to-end integration testing. However, real APIs always have some latency and make your tests slower to run.



You can alternatively use a **mock API**. This is common if the developer doesn't have access to the real API. Using a mock API requires creating and maintaining the mock API implementation. There are several libraries that you can use for this with React, such as **msw (Mock Service Worker)** and **nock**.

Let's begin:

1. Create a new file called `Carlist.test.tsx` in your `src` folder. We will import the `Carlist` component and render it. The component renders the `'Loading...'` text when data from the backend is not available yet. The starter code looks like the following:

```
import { describe, expect, test } from 'vitest'; import
{ render, screen } from '@testing-library/react'; import
'@testing-library/jest-dom/vitest'; import Carlist from
'./components/Carlist';

describe("Carlist tests", () => {   test("component renders",
() => {       render(<Carlist />);
expect(screen.getByText(/Loading/i)).toBeInTheDocument();
})
});
```

2. Now, if you run your test cases, you will get the following error: **No QueryClient set, use QueryClientProvider to set one**. We used React Query for networking in our

Carlist component; therefore, we need `QueryClientProvider` in our component. The source code below shows how we can do that. We have to create a new `QueryClient` and set retries to false. By default, React Query retries queries three times, which might cause timeouts in your test case if you want to test error cases:

```
import { QueryClient, QueryClientProvider } from
  '@tanstack/react-query'; import { describe, test }
from 'vitest'; import { render, screen } from
  '@testing-library/react'; import
  '@testinglibrary/jest-dom/vitest'; import
  Carlist from
  './components/Carlist';
const queryClient = new QueryClient({
  defaultOptions: { queries: {
    retry: false,
  }},
});
const wrapper = ({ children } : { children:
  React.ReactNode }) => (
  <QueryClientProvider client
    = {
  queryClient}>{children} </QueryClientProvider>);
describe("Carlist tests", () => { test("component
  renders", () => { render(<Carlist />, ); { wrapper }
  expect(screen.getByText(/Loading/i)).toBeInTheDocument();
  })
});
```

We also created a wrapper that returns a `QueryClientProvider` component. Then, we used the render function's second argument and passed our wrapper, which is a React component, so that the wrapper wraps the `Carlist` component. This is useful functionality when you want to wrap your component with additional wrappers. The final result is that the `Carlist` component is wrapped inside the `QueryClientProvider`.

3. Now, if you rerun your tests, you won't get an error and your new test case will pass.

The test run now includes two test files and two tests:

```

✓ src/App.test.tsx (1)
✓ src/Carlist.test.tsx (1)

Test Files  2 passed (2)
Tests       2 passed (2)
Start at    13:06:52
Duration    4.75s (transform 130ms, setup 987ms,
PASS  Waiting for file changes ...
press h to show help, press q to quit

```

Figure 15.2: Test run

4. Next, we will test that our `getCars` fetch is invoked and the cars are rendered in the data grid. The network calls are asynchronous, and we don't know when the response will arrive. We will use the React Testing Library's `waitFor` function to wait until the **NEW CAR** button is rendered because then we know that the network request has succeeded. The test will proceed after the condition is met.

Finally, we will use a matcher to check that the Ford text can be found in the document. Add the following highlighted import to the `Carlist.test.tsx` file:

```
import { render, screen, waitFor } from '@testing-library/react';
```

5. The test looks like the following:

```

describe("Carlist tests", () => {
  test("component renders", () => {
    render(<Carlist />, { wrapper });
    expect(screen.getByText(/Loading/i))
      .toBeInTheDocument();
  })
  test("Cars are fetched", async () => {
    render(<Carlist />, { wrapper });
    await waitFor(() => screen.getByText(/New Car/i));
    expect(screen.getByText(/Ford/i)).toBeInTheDocument();
  })
});

```

6. If you rerun the tests, you can see that three tests pass now:

```

✓ src/App.test.tsx (1)
✓ src/Carlist.test.tsx (2) 489ms

Test Files  2 passed (2)
Tests       3 passed (3)
Start at    13:57:29
Duration    3.88s

PASS Waiting for file changes...
press h to show help, press q to quit

```

Figure 15.3: Test run

We have now learned the basics of Vitest and how to create and run test cases in a Vite React app.

Next, we will learn how to simulate user actions in our test cases.

Firing events in tests

The React Testing Library provides a `fireEvent()` method that can be used to fire DOM events in your test cases. The `fireEvent()` method is used in the following way. First, we have to import it from the React Testing Library:

```
import { render, screen, fireEvent } from '@testing-library/react';
```

Next, we have to find the element and trigger its event. The following example shows how to trigger an input element's change event and a button's click event:

```

// Find input element by placeholder text
const input = screen.getByPlaceholderText('Name');

// Set input element's value
fireEvent.change(input, {target: {value: 'John'}});

// Find button element by text
const btn = screen.getByText('Submit');

// Click button
fireEvent.click(btn);

```

After the events are triggered, we can assert the expected behavior.

There is also a companion library for the Testing Library that is called `user-event`. The `fireEvent` function triggers element events, but browsers do more than only triggering one

event. For example, if a user types some text into an input element, it is first focused, and then keyboard and input events are fired. `user-event` simulates the full user interaction.

To use the `user-event` library, we have to install it in our project with the following npm command:

```
npm install -D @testing-library/user-event
```

Next, we have to import `userEvent` in the test file:

```
import userEvent from '@testing-library/user-event';
```

Then, we can create an instance of `userEvent` using the `userEvent.setup()` function. We can also call the API directly, which will call `userEvent.setup()` internally, and this is how we will use it in the following examples. The `userEvent` provides multiple functions to interact with the UI, such as `click()` and `type()`:

```
// Click a button await
userEvent.click(element);

// Type a value into an input element await
userEvent.type(element, value);
```

As an example, we will create a new test case that simulates a **NEW CAR** button press in our `Carlist` component and then checks that the modal form is opened:

1. Open the `Carlist.test.tsx` file and import `userEvent`:

```
import userEvent from '@testing-library/user-event';
```

2. Create a new test inside the `describe()` function where we have our `Carlist` component tests. In the test, we will render the `Carlist` component and wait until the **NEW CAR** button is rendered:

```
test("Open new car modal", async () => {
  render(<Carlist />, { wrapper });

  await waitFor(() => screen.getByText(/New Car/i)); })
```

3. Then, find the button using the `getByText` query and use the `userEvent.click()` function to press the button. Use a matcher to verify that the **SAVE** button can be found in the document:

```
test("Open new car modal", async () => {
  render(<Carlist />, { wrapper });

  await waitFor(() => screen.getByText(/New Car/i));
  await userEvent.click(screen.getByText(/New Car/i));

  expect(screen.getByText(/Save/i)).toBeInTheDocument()
;
})
```

4. Now, rerun your tests and see that four test cases pass:

```
✓ src/App.test.tsx (1)
✓ src/Carlist.test.tsx (3) 674ms

Test Files 2 passed (2)
Tests 4 passed (4)
Start at 14:35:58
Duration 4.06s

PASS Waiting for file changes...
press h to show help, press q to quit
```

Figure 15.4: Test run

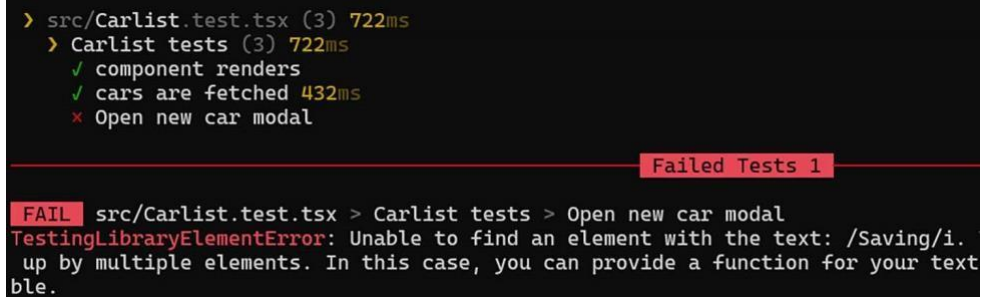
We can use the `getByRole` query to find elements based on their roles, such as buttons, links, and so on. Below is an example of how to find a button that contains the text `Save` by using the `getByRole` query. The first argument defines the role, and the second argument's `name` option defines the button text:

```
screen.getByRole('button', { name: 'Save' });
```

5. We can also test how a failed test looks by changing the text in the test matcher, for example:

```
expect(screen.getByText(/Saving/i)).toBeInTheDocument();
```

Now, if we rerun the tests, we can see that one test fails, along with the reason for the failure:



```
> src/Carlist.test.tsx (3) 722ms
> Carlist tests (3) 722ms
  ✓ component renders
  ✓ cars are fetched 432ms
  ✗ Open new car modal

Failed Tests 1

FAIL src/Carlist.test.tsx > Carlist tests > Open new car modal
TestingLibraryElementError: Unable to find an element with the text: /Saving/i.
up by multiple elements. In this case, you can provide a function for your text
ble.
```

Figure 15.5: Failed test

Now, you know the basics of testing user interactions in your React components.

End-to-end testing

End-to-end (E2E) testing is a methodology that focuses on testing an entire application's workflow. We will not cover it in detail in this book, but we will give you an idea about it and cover some tools that we can use.

The goal is to simulate user scenarios and interactions with the application to make sure that all components work together correctly. E2E testing covers frontend, backend, and all interfaces or external dependencies of the software that is being tested. The E2E testing scope can also be *cross-browser* or *cross-platform*, where an application is tested using multiple different web browsers or mobile devices.

There are several tools available for end-to-end testing, such as:

- **Cypress** (<https://www.cypress.io/>): This is a tool that can be used to create E2E tests for web applications. Cypress tests are simple to write and read. You can see your application's behavior during the test execution in the browser and it also helps you to debug if there are any failures. You can use Cypress for free, with some limitations.
- **Playwright** (<https://playwright.dev/>): This is a test automation framework designed for E2E testing, and it is developed by Microsoft. You can get a Visual Studio Code extension for Playwright and start to use it in your project. The default language for writing tests with Playwright is TypeScript, but you can also use JavaScript.

E2E testing helps to verify that your application meets its functional requirements.